From Sparse Ratings to Top-10 Recommendations: Matrix Factorization on MovieLens 100K

Student: Anna Grishkina
Team: Individual
Date: 2026-10-14
Assignment: A05 — From Ratings to Recommendations (Week 5)
Live app: https://af851574-oss.github.io/RecSys-LLMs/week5/

Abstract

MovieLens 100K has 100,000 observed ratings in a user–movie matrix that is 93.70% unobserved, so a recommender must learn from known entries without treating missing ratings as zeros. I trained 16-dimensional user and movie factors in TensorFlow.js, split each user's ratings chronologically, and selected model weights by validation RMSE. In the final browser run, test RMSE was **1.0472** against **1.2289** for the training-mean baseline; macro Recall@10 was **0.0554** among 861 eligible users. The starter's default regularization collapsed the factors to zero, while reducing that single hyperparameter on validation recovered useful predictions but left discovery and score-calibration limitations.

Index Terms — matrix factorization, MovieLens, recommendation, RMSE, Recall@10

1. Introduction

Problem. Given a user and an unrated movie, estimate a 1–5 rating and rank candidate movies. MovieLens 100K [1], [2] has 943 users, 1,682 movies and 100,000 ratings, so 93.6953% of the logical 943 × 1,682 matrix is unobserved. A missing rating is not a dislike or a numeric zero.

Why it matters. Pairwise similarity becomes unstable with little overlap. Matrix factorization learns compact user and movie vectors from *observed* pairs, giving a score even if two users never rated the same film [3]. Rating error and recommendation coverage answer different questions, so I report both RMSE and Recall@10 rather than treating one as a substitute for the other.

Example. At recommendation time user 1 had rated 244 movies in train/validation. *Contact (1997)* was not in that history, appeared in the Top-10, and was later rated 5 in the held-out test split; that is a verifiable hit, not proof that an online promotion would cause a viewing.

Contributions.

- Implemented TensorFlow.js factor training, validation-weight restoration, catalog Top-10 and strict training-candidate Recall@10 in the provided browser app.
- Found a degenerate default setting, tuned one variable using validation, and corrected an impossible rank-11 test fixture without changing the dataset or page UI.

2. Related Work

Koren, Bell and Volinsky [3] describe latent-factor recommendations learned from sparse observed ratings. Unlike classical SVD on a fully specified matrix, this task fits only observed entries; I did not fill missing cells with zero [3], [4]. Harper and Konstan [4] document the MovieLens dataset, while GroupLens [2] supplies the original 100K release. The instructor's starter [1] specifies TensorFlow.js mini-batches and the training/test protocol; I used the TensorFlow.js training API [5] rather than a per-rating JavaScript optimizer. User-neighbor collaborative filtering (Week 3) was an alternative, but its co-rating similarity can be weak in a sparse matrix; a feature-based two-tower model from the lecture [6] is a later alternative for cold-start items but needs features and a separate training design.

3. Method

Data and split. The provided `data.js` parses `u.data` and `u.item` and sorts each user's ratings by timestamp, breaking ties by movie id. For a user with n ratings it assigns `floor(0.8n)` to train, `floor(0.1n)` to validation and the remainder to test. The resulting sizes are **79,619 / 9,596 / 10,785**; none of the 943 users is dropped. There are 1,611 movie ids in training; the other 71 catalog ids lack learned training evidence. The allocated factor tensors include unused index zero, hence shapes **944 × 16** (users) and **1,683 × 16** (movies), not the logical matrix dimensions.

Model. The train mean is μ = **3.580603**. For observed training pairs only, the model computes `μ + p_u · q_i` using 16-dimensional trainable vectors; scalar rating predictions are clipped to [1,5]. Small random initial factors (standard deviation 0.05) are trained by shuffled Adam mini-batches (size 64, learning rate 0.01) on mean squared error plus an L2 penalty on factor matrices. Factor-gradient updates use `tf.gather`, elementwise multiplication/sum and `optimizer.minimize`, not a JavaScript loop over rating-by-rating optimizer calls. Snapshots of the lowest-validation-RMSE epoch are restored before test evaluation. Random initialization and shuffling are not seeded, so separate runs do not have identical metrics.

Evaluation. RMSE is `sqrt(mean((prediction − rating)^2))` over a split; the baseline predicts μ for every test pair. For user u, relevant test movies have rating ≥ 4 and must be in the **global training-movie pool**. The evaluation ranks only that pool, excluding movies the user rated in train or validation; `recall_u = hits@10 / eligible relevant`

`test movies`. Overall Recall@10 is the **mean over evaluable users**, not total hits divided by total relevant items. The ordinary Top-10 button ranks the full catalog after the same seen-item exclusion, so its candidate set is intentionally broader than the evaluation pool.

Environment. Plain HTML/CSS/JavaScript, TensorFlow.js **4.22.0** [5], Node.js **24.10.0** for checks, headless Chrome **155.0.8059.41** through DevTools Protocol; the measured training backend was **WebGL**, not a claimed server GPU. The app needs HTTP to fetch the two MovieLens files and initial network access for its TensorFlow.js CDN script. No build step or Python package is required by the published app.

4. Experiments

Protocol and hyperparameters. I first reproduced the starter's harness **4 passed / 0 failed / 7 pending**. The page's untouched default is 20 epochs, learning rate 0.01, L2 **0.05** and batch size 64. I varied only L2 using *validation* RMSE; the final `0.0001` setting is entered in the page's L2 input before pressing Train. Each row below is an actual full 20-epoch browser run; numbers vary across random initializations.

Training run	L2	Best epoch	Validation RMSE	Test RMSE	Comparison
Mean-only baseline	—	—	1.1830	**1.2289**	reference
Starter default	0.05	1	1.1830	1.2289	collapsed to mean
Selection trial	0	1	1.0207	1.0613	better
Selection trial	**0.0001**	20	**0.9990**	1.0498	selected L2 by validation
Post-fix click 1	0.0001	20	1.0037	1.0535	fresh random run
Post-fix click 2 (final model)	**0.0001**	**14**	**0.9959**	**1.0472**	**used for Top-10 and Recall**

The final test RMSE improves on the baseline by **0.1817** (1.2289 − 1.0472). The second click tested repeat-training behavior, not another hyperparameter; its lower test RMSE was not used to choose L2. After both clicks, history contained exactly 20 rows and `computeRMSEFor(validation)` matched the selected snapshot's recorded validation RMSE; tensor count stayed 27 → 27.

Retrieval. From 1,611 training-set movie candidates, 861 users have at least one eligible test movie rated ≥ 4; 82 test users have none. Thirteen *distinct* relevant test movie ids never occur in training and are excluded. In the final model, there were **326 hits among 5,142 eligible relevant user–movie pairs**; macro Recall@10 was **0.0554** (the pooled 326/5,142 = 0.0634 is a *different*, micro-averaged statistic). User 1 had **3 hits among 19 eligible relevant movies = 0.1579**: *Contact*, *The Wrong Trousers* and *Fargo* were each rated 5 later; *Good Will Hunting* appeared in the list but was rated 3 in test. User 405 has no relevant test movie and is excluded from the Recall average, though the user has a test split and a Top-10 list.

Checkable calculations. In a hand-sized factor fixture, μ = 3, user vector (1,2), item vector (3,4) gives `3 + (1×3 + 2×4) = 14`, clipped to **5**. A two-rating RMSE fixture with predictions (4,2) and ratings (3,4) gives `sqrt(((4−3)^2+(2−4)^2)/2) = sqrt(2.5) = 1.581139`. Independent parsing of the raw ratings reproduced the split sizes, μ, baseline RMSE, 93.6953% sparsity, 13 cold relevant ids, 82 excluded users and user 1's 3/19 hit count.

Executed tests. The final Node check and the page's own harness both pass **11/11**. Extra regressions check earliest-epoch ties even with unsorted history, a strict train-only candidate Top-1, and repeated Train history reset. A real HTTP/CDP browser run clicked Run tests, Train twice, Recommend for both users and Evaluate Recall; it captured the backend, checkpoint weights/RMSE, rendered lists and Recall. `data.js`, `index.html`, `style.css`, `u.data`, `u.item` and the instructor readme stayed byte-identical to upstream.

5. Discussion

Familiar choices vs. new discoveries (the single business question). User 1's train/validation history is heavy in Drama (93 titles), Comedy (81) and Action (69); five-star examples include *Monty Python and the Holy Grail*, *The Empire Strikes Back* and *Dead Poets Society*. Their Top-10 has *Contact* (score 5.40), *Casablanca* (5.19), *A Close Shave* (5.15), *Lawrence of Arabia* (5.04), *The Wrong Trousers* (5.03), *Dr. Strangelove* (5.02), *Fargo* (4.99), *Titanic* (4.95), *One Flew Over the Cuckoo's Nest* (4.95) and *Good Will Hunting* (4.91). All were unseen *at recommendation time*, but four later appear in the held-out test ratings; the three five-star hits support the relevance of familiar prestige films, while old classics and two Aardman animations are plausible discovery opportunities, not evidence that the user actually watched them.

User 405's train/validation ratings average **1.921** across 662 films, with many Drama (257), Comedy (215) and Action (118) titles; past five-star picks include *Scream*, *The Shawshank Redemption* and *Raiders of the Lost Ark*. Their Top-10 instead starts with *Boogie Nights* (raw score 6.70), *Die Hard* (5.68), *Face/Off* (5.38), *Liar Liar* (5.29), *Willy Wonka and the*

Chocolate Factory (5.17), then *Hearts and Minds*, *Someone Else's America*, *The Fifth Element*, *Mighty Aphrodite* and *Les Boys*. It mainly stays in familiar Drama/Action/Comedy territory, with a few less obvious titles. The business value is low-risk, recognizable selections for retention; the downside is little taste expansion, and the extreme scores for a harsh rater make the ranking unfit to display as predicted star ratings. I would add calibrated outputs and test a diversity-aware reranker before claiming discovery gains.

Failure, root cause, fix and check. With the starter default L2 = 0.05, train/val RMSE froze at **1.1052 / 1.1830**, test RMSE equaled the baseline, and all recommendation scores became μ. The specified full-matrix penalty is applied to every one of about **1,245 mini-batches per epoch**, while each observed rating contributes to one batch per epoch: at 0.05 the shrinkage overwhelms the data gradient. I changed **one** variable, L2 to **0.0001**, selected it by validation, and observed test RMSE **1.0472** in the final run, with restored validation checkpoint weights. This was a configuration fix through an existing input, not a silent change to the loss formula or to the starter's default UI.

Other defects found by review. A second Train click initially appended history from the first (40 rows and possibly a stale checkpoint); resetting history at the start of `trainMF` now gives 20 rows each time and zero net tensor growth. The starter's rank-11 Recall fixture had only three train movie ids, making strict train-only ranking mathematically unable to reach rank 11. I minimally added the other nine catalog ids to that fixture's training pool and restricted *evaluation* ranking to train movie ids; the 11 checks still pass. The general Top-10 keeps the full catalog as the starter specifies. A readme code example also chains `.dispose()` onto `optimizer.minimize(..., false)`, which actually returns no tensor; the implementation uses the correct API semantics [5].

What worked: batched tensor gradients trained 16-dimensional factors in a static browser page; a validation-restored checkpoint beats the baseline; real click tests caught issues that the toy harness missed. **What did not:** default L2 makes the model degenerate, and unbounded ranker scores can exceed five although scalar predictions clamp. **Next improvement:** normalize L2 per touched batch/user/item, then retune on validation and test whether a bias-aware or diversity-aware ranking improves held-out Recall and discovery without harming RMSE. No later-period causal impact or click-through uplift was measured.

6. AI Usage Disclosure

Tools. Pi Coding Agent **0.85.1**, with a separate task-only implementation subagent using **GLM-5.3 via Z.ai**. It generated code, performed TensorFlow.js API checks, ran Node and headless-Chrome experiments, and maintained `week5/prompt.md`; an AI-assisted review pass independently checked the raw-data counts, source logic, reference records and the 3/19 example, then corrected the repeated-Train and candidate-pool issues. AI assistance also helped draft this report and the demo script.

Verification and limits. The checks in Sections 4–5 were executed, not inferred from a language-model description; browser logs and small runnable checks underpin their numbers. These were **AI-assisted independent checks**, not a claim of unaided human calculation. I did not independently verify TensorFlow.js kernel internals or claim that headless WebGL timings predict every viewer's machine. The original implementation session completed; a resumed verification session later ended with a tooling failure while updating README, after its post-fix browser logs and journal had already been saved. The reported results come from that successful saved evidence, not from a claimed successful exit of the resumed session.

Session artifact. `argrishkina_a05_session.json` is a **privacy-redacted derived audit** from the task-only worker session, containing actual tool calls, file-change descriptions and verified milestones while excluding conversation, credentials and local account identifiers. It is **not an unmodified Pi export**. The homework guidelines request an unmodified tool-native export; this privacy choice may not meet that rule, and the report does not claim otherwise.

References

[1] S. Jin, "HW5 — From Ratings to Recommendations (week5)," *RecSys-LLMs*, GitHub, 2026. [Online]. Available: https://github.com/dryjins/RecSys-LLMs/tree/main/week5. [Accessed: 2026-10-14].

[2] GroupLens, "MovieLens 100K Dataset," University of Minnesota. [Online]. Available: https://grouplens.org/datasets/movielens/100k/. [Accessed: 2026-10-14].

[3] Y. Koren, R. Bell, and C. Volinsky, "Matrix Factorization Techniques for Recommender Systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009, doi: 10.1109/MC.2009.263.

[4] F. M. Harper and J. A. Konstan, "The MovieLens Datasets," *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, pp. 1–19, 2015, doi: 10.1145/2827872.

[5] TensorFlow.js, "Train models," TensorFlow.js Guide. [Online]. Available: https://www.tensorflow.org/js/guide/train_models. [Accessed: 2026-10-14].

[6] S. Jin, "Matrix Factorization & Two Tower," LLM4Rec lecture slides, HSE University, 2026, slides 4, 6–9 and 11–12.